

uFPGA-Emu: Ensino de Circuitos Digitais e Arquitetura de Computadores em Microcontroladores de Baixo Custo

Resumo do projeto desenvolvido — apresentação interna ao grupo de pesquisa

João Pedro Hallack Sansão

13 de agosto de 2026

DTECH – Universidade Federal de São João del-Rei (UFSJ), Campus Alto Paraopeba

Custo de FPGAs educacionais

Placas FPGA educacionais custam de US\$ 25 a US\$ 130; um laboratório com 40 alunos passa de US\$ 5.200 (ex.: DE10-Lite).

Simuladores em PC resolvem o custo, mas...

A atividade fica em ambiente **abstrato**: sem LEDs, botões, protoboard e **experiência física**.

Lacuna pedagógica

Entre software puro e hardware dedicado caro, falta um degrau de entrada barato com interação física real.

“Traduzir automaticamente Verilog sintetizável para C e executar o circuito em microcontroladores de baixo custo.”

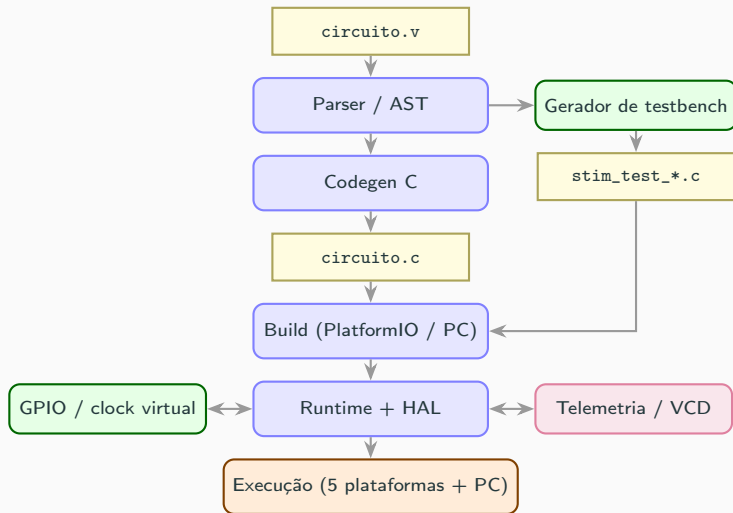
Plataformas alvo

- ESP32 (US\$ 3–5) RP2040 (US\$ 1)
- ESP8266 D1 Mini (US\$ 3)
- Arduino Due (US\$ 10)
- Raspberry Pi 3 B+ (US\$ 35)

Experiência física

- LEDs, botões e buzzers reais
- Montagem em protoboard
- Telemetria serial em tempo real
- Kit por aluno < US\$ 10

Arquitetura: pipeline Verilog → C → execução



Toolchain em Python (`pc_tool`)

Parser + Codegen

- Lexer manual + parser recursivo descendente
→ AST
- always com posedge/negedge, if/else, case, for, repeat
- generate aninhado, task/function, instanciação de submódulos (flatten)
- Reduction, ternário, concatenação, part-select, ===

Otimizações

- Dead register elimination
- Constant folding e $\ll 0$ suppression
- Detecção de borda global

Tempo de geração (laptop i5, Python 3.11)

Circuito	Tempo
blinky (15 linhas)	12 ms
alu (62)	25 ms
uart_tx (96)	35 ms
tiny_cpu (124)	42 ms

3 mil linhas de Verilog/s.

Verilog (18 linhas)

```
// counter.v - contador de 4 bits
module counter (
    input wire clk,
    input wire rst,
    output reg [3:0] count
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            count <= 4'b0;
        else
            count <= count + 4'b1;
    end
endmodule
```

C gerado (semântica preservada)

```
void circuit_eval(model_state_t *state, uint32_t inputs, uint32_t *outputs)
{
    uint32_t clk = GET_BIT(inputs, 0);
    uint32_t rst = GET_BIT(inputs, 1);
    uint8_t clk_edge = (clk == 1) && (state->regs[1] != 1);
    state->regs[1] = (clk) & 0x1;
    if (rst) {
        state->regs[0] = (0) & 0xF;
    } else if (clk_edge) {
        state->regs[0] = ((state->regs[0] + 1)) & 0xF;
    }
    *outputs = ((state->regs[0] & 0xF) << 0);
}
```

- clk_edge: detecção de borda = posedge
- Guard rst || clk_edge: bordas de rst e clk

Clock virtual

- Timer interno alterna o sinal de clock
- Sem sinal externo (portátil p/ qualquer MCU)

Loop de emulação (emulator_step)

1. Lê entradas (GPIO físico ou virtual)
2. `circuit_eval()` um ciclo
3. Escreve saídas; incrementa o clock virtual

Comandos via serial

- `vset <pin> <val>` — altera input em tempo real
- Telemetria: `CLK=.. IN=0x.. OUT=0x.. REG0=0x..`

HAL — abstração por plataforma

- `hal_gpio`, `hal_timer`, `hal_serial`, `hal_mutex`
- Portas: ESP32, RP2040, Due, ESP8266, RPi 3 B+ e PC

VCD opcional

- Traces para visualização no GTKWave (IEEE 1364-2001)

Testbenches gerados

- CSV de estímulo → `stim_test_*.c`
- Auto-assertions, execução em PC sem hardware

5 plataformas, 26 circuitos cada

- **ESP32** — Xtensa LX6, 240 MHz
- **RP2040** — Cortex-M0+, 133 MHz
- **Arduino Due** — Cortex-M3, 84 MHz
- **ESP8266** (Wemos D1 Mini) — Xtensa LX106, 80 MHz
- **RPi 3 B+** — Cortex-A53, 1,2 GHz

Circuitos de portas lógicas a **ULA de 8 bits** e **CPU completa de 8 bits** (`tiny_cpu`).

102 testes automatizados

- Parser e codegen (unitários)
- Testbenches PC com CSV de estímulo
- Testbenches de integração (compile + run)
- 100% de aprovação

Avaliação

Técnica, não pedagógica. Estudo controlado com alunos fica como trabalho futuro.

Resultados: clock virtual por plataforma

Circuito	ESP32	RP2040	ESP8266	Pi 3 B+
	kHz	kHz	kHz	kHz
not_keyword	–	–	64,8	–
blinky	95,2	90,9	–	2954
mux	94,8	–	–	2693
counter	84,0	99,6	–	2230
fsm_101	58,6	–	–	2489
alu	38,4	–	–	–
tiny_cpu	15,0	53,2	25,6	999

- Clock virtual decai com a complexidade combinacional
- Pi 3 B+ é $\sim 10\text{--}30\times$ mais rápido que os MCUs
- Faixa geral: 15 kHz (CPU no ESP32) a 3 MHz (circuitos simples no Pi)
- Valores de referência de ponto único (mesmo toolchain/firmware)

Característica	Simulador PC	FPGA	uFPGA-Emu
Custo por estação	Grátis	US\$ 25–130	US\$ 1–5
Experiência física	Não	Sim	Sim
Tempo de compilação	Segundos	Minutos	Segundos
Curva de aprendizado	Baixa	Alta	Baixa
Clock máximo	Ilimitado	MHz–GHz	~25 kHz–3 MHz
Verilog automático	–	Síntese manual	Automático

Laboratório com 40 alunos

Menos de US\$ 400 com ESP32 ou US\$ 100 com RP2040, contra mais de US\$ 5.200 com placas FPGA educacionais.

Estado atual

- Paper submetendo à **IJCAE** (10 páginas)
- 26 circuitos validados em 5 plataformas
- 102 testes automatizados passando
- Código aberto no GitHub

Próximos passos

- Estudo controlado com alunos (SUS + teste A/B)
- Emular CPU **RISC-V** a partir do Verilog
- Port para ATmega328P (análise já feita)
- Múltiplos domínios de clock
- Otimizar `circuit_eval()` (mais kHz)

Obrigado!

- **Contato:** joao@ufsj.edu.br
- **ORCID:** 0000-0003-0095-2629
- **Código:** github.com/jsansao/ufpga-emu

Convite à colaboração

Interessados em **portar para novas plataformas**, colaborar no **estudo pedagógico** ou testar a ferramenta em disciplinas, entrem em contato.